2D Transformation in OpenGL

Simple Square → Scale → Translate2d → Rotate → Transformation2d

Computer Graphics - Tutorial 2

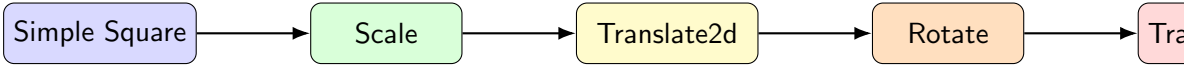
Aim of This Presentation

Main Aim

To understand how a simple 2D square is drawn and then transformed using scaling, translation, and rotation.

- First, we draw a simple square.
- Then, we scale the square.
- Then, we translate the square.
- Then, we rotate the square.
- Finally, we combine transformations in one program.

Overall Flow



Meaning: Each step changes the square in a different way.

OpenGL Basic Idea

- OpenGL is used to draw graphics on the screen.
- GLUT creates the display window.
- `glBegin(GL_QUADS)` starts drawing a four-sided object.
- `glVertex2f(x,y)` gives the coordinate of each vertex.
- `glEnd()` finishes the drawing.

Common Import Section

This section imports OpenGL libraries.

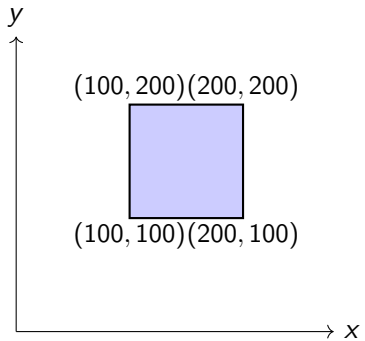
```
from OpenGL.GL import *
from OpenGL.GLUT import *
from OpenGL.GLU import *
```

- `OpenGL.GL` gives drawing functions.
- `OpenGL.GLUT` creates and controls the window.
- `OpenGL.GLU` provides utility functions.

1. Simple Square

A square is drawn using four points.

$(100, 100), \quad (200, 100), \quad (200, 200), \quad (100, 200)$



Simple Square Code

```
def square():
    # bottom left, bottom right, top right, top left
    glBegin(GL_QUADS)
    glVertex2f(100, 100)
    glVertex2f(200, 100)
    glVertex2f(200, 200)
    glVertex2f(100, 200)
    glEnd()
```

Meaning

This function draws one square using four vertices.

Which Line Draws Which Corner?

Code Line	Meaning
`glVertex2f(100,100)`	Bottom-left point
`glVertex2f(200,100)`	Bottom-right point
`glVertex2f(200,200)`	Top-right point
`glVertex2f(100,200)`	Top-left point

Window Setup Code

```
w,h = 500,500

glutInit()
glutInitDisplayMode(GLUT_RGBA)
glutInitWindowSize(w, h)
glutInitWindowPosition(0, 0)
wind = glutCreateWindow(b"OpenGL Coding Practice - Simple Square")
glutDisplayFunc(showScreen)
glutIdleFunc(showScreen)
glutMainLoop()
```

Meaning

This part creates a 500×500 window and continuously displays the square.

2D Coordinate Setup

```
def iterate():
    glViewport(0, 0, 500, 500)
    glMatrixMode(GL_PROJECTION)
    glLoadIdentity()
    glOrtho(0.0, 500, 0.0, 500, 0.0, 1.0)
    glMatrixMode(GL_MODELVIEW)
    glLoadIdentity()
```

- `glViewport()` sets the drawing area.
- `glOrtho()` creates a 2D coordinate system.
- Here, x and y values go from 0 to 500.

Display Function

```
def showScreen():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
    iterate()
    glColor3f(1.0, 0.0, 3.0)
    square()
    glutSwapBuffers()
```

Meaning

This function clears the screen, sets the coordinate system, selects color, and draws the square.

2. Scaling

Scaling means changing the size of an object.

$$x' = x \times S_x$$

$$y' = y \times S_y$$

If the scale factor is 1.5, the object becomes 1.5 times larger.

Scale Matrix Code

```
def get_scale2d(factor):
    R = np.array([[factor, 0, 0],
                  [0, factor, 0],
                  [0, 0, 1]], dtype=np.float32)
    return np.transpose(R)
```

Responsible Code

`factor` is responsible for increasing or decreasing the size.

Scale Matrix Meaning

$$S = \begin{bmatrix} factor & 0 & 0 \\ 0 & factor & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

For `factor = 1.5`:

$$x' = 1.5x, \qquad y' = 1.5y$$

So the square becomes larger.

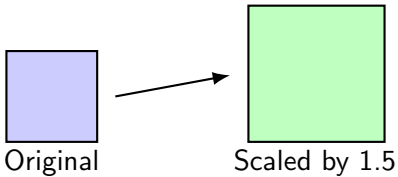
Scale Application Code

```
pc = np.array([[100, 100, 1],
               [200, 100, 1],
               [200, 200, 1],
               [100, 200, 1]], dtype=np.float32)

pc_new = np.array([i @ get_scale2d(1.5) for i in pc],
                  dtype=np.float32)
```

- pc stores the original square points.
- `get_scale2d(1.5)` creates scaling matrix.
- `i @ get_scale2d(1.5)` applies scaling to each point. means Take every point i from pc, scale that point by 1.5, and create a new list of the scaled points.

Scaling Visual Meaning



3. Translation

Translation means moving the object from one place to another.

$$x' = x + T_x$$

$$y' = y + T_y$$

In your code:

$$T_x = 200, \qquad T_y = 150$$

So the square moves right by 200 and up by 150.

Translate2d Matrix Code

```
def get_translate2d(tx, ty):
    R = np.array([[1, 0, tx],
                  [0, 1, ty],
                  [0, 0, 1]], dtype=np.float32)
    return np.transpose(R)
```

Responsible Code

`tx` moves the square horizontally, and `ty` moves it vertically.

Translate2d Matrix Meaning

$$T = \begin{bmatrix} 1 & 0 & T_x \\ 0 & 1 & T_y \\ 0 & 0 & 1 \end{bmatrix}$$

For your code:

$$T = \begin{bmatrix} 1 & 0 & 200 \\ 0 & 1 & 150 \\ 0 & 0 & 1 \end{bmatrix}$$

This moves every point by $(200, 150)$.

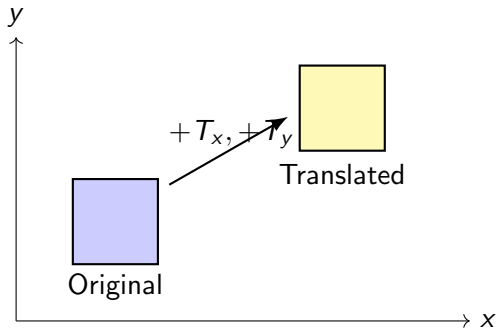
Translate2d Application Code

```
pc = np.array([[100, 100, 1],
               [200, 100, 1],
               [200, 200, 1],
               [100, 200, 1]], dtype=np.float32)

pc_new = np.array([i @ get_translate2d(200,150) for i in pc],
                  dtype=np.float32)
```

- Original square remains at original place.
- New square is shifted by $(200, 150)$.

Translation Visual Meaning



4. Rotation

Rotation means turning an object around a point.

$$x' = x\cos\theta - y\sin\theta$$

$$y' = x\sin\theta + y\cos\theta$$

In your code, the angle is:

$$-15^\circ$$

Negative angle means clockwise rotation.

Rotate Matrix Code

```
def get_rotate2d(angle):
    rad = np.radians(angle)
    R = np.array([[np.cos(rad), np.sin(rad), 0],
                  [-np.sin(rad), np.cos(rad), 0],
                  [0, 0, 1]], dtype=np.float32)
    return np.transpose(R)
```

Responsible Code

`np.radians(angle)` converts degree into radian because sine and cosine use radians.

Rotate Matrix Meaning

$$R = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- `cos(rad)` controls horizontal rotation effect.
- `sin(rad)` controls vertical rotation effect.
- Angle decides how much the square turns.

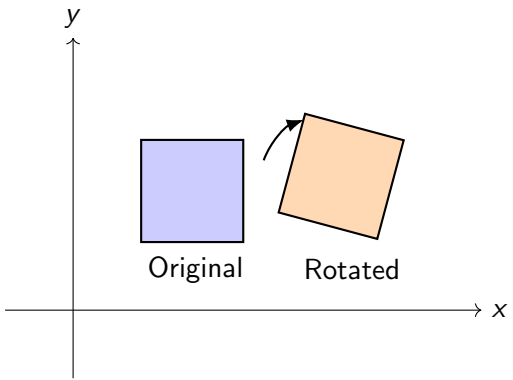
Rotate Application Code

```
pc = np.array([[100, 100, 1],
               [200, 100, 1],
               [200, 200, 1],
               [100, 200, 1]], dtype=np.float32)

pc_new = np.array([k @ get_rotate2d(-15) for k in pc],
                  dtype=np.float32)
```

- `get_rotate2d(-15)` creates rotation matrix.
- Each point is multiplied with this matrix.
- Result is the rotated square.

Rotation Visual Meaning



5. Transformation2d

Meaning

`transformation_2d.py` shows how OpenGL built-in transformation functions can be applied to a rectangle.

It uses:

- `glTranslatef()` for translation.
- `glScalef()` for scaling.
- `glRotatef()` for rotation.
- `glPushMatrix()` and `glPopMatrix()` to protect transformation state.

Rectangle Drawing Function

```
def draw_rect(x, y, width, height):
    glBegin(GL_QUADS)
    glVertex2f(x, y)
    glVertex2f(x + width, y)
    glVertex2f(x + width, y + height)
    glVertex2f(x, y + height)
    glEnd()
```

Meaning

This function draws a rectangle from starting point (x, y) using width and height.

Transformation2d Main Drawing Code

```
def draw():
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
    glLoadIdentity()
    iterate(width, height)

    glColor3f(0.0, 0.0, 1.0)
    glPushMatrix()

    draw_rect(10, 10, 200, 100)

    glTranslatef(300,200,0)
    glScalef(2,2,0)
    glRotatef(45,0,0,1)

    draw_rect(10, 10, 200, 100)

    glPopMatrix()
    glutSwapBuffers()
```

Line-by-Line Meaning of Transformation2d

Code	Meaning
`draw_rect(10,10,200,100)`	Draws original rectangle.
`glTranslatef(300,200,0)`	Moves rectangle right by 300 and up by 200.
`glScalef(2,2,0)`	Enlarges the rectangle 2 times in x and y directions.
`glRotatef(45,0,0,1)`	Rotates rectangle by 45° around z-axis.
`draw_rect(10,10,200,100)`	Draws the transformed rectangle.

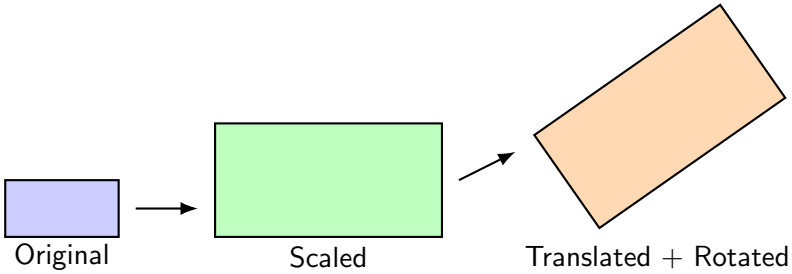
Why glPushMatrix and glPopMatrix?

- OpenGL transformations change the current transformation matrix.
- `glPushMatrix()` saves the current matrix.
- After applying transformations, `glPopMatrix()` restores the old matrix.
- This prevents transformations from affecting other objects.

Simple Meaning

Use `glPushMatrix()` before transformation and `glPopMatrix()` after transformation.

Transformation2d Visual Flow



Manual Matrix Method vs OpenGL Built-in Method

Operation	Manual Matrix Code	OpenGL Built-in Code
Scaling	`get_scale2d(1.5)`	`glScalef(2,2,0)`
Translation	`get_translate2d(200,150)`	`glTranslatef(300,200,0)`
Rotation	`get_rotate2d(-15)`	`glRotatef(45,0,0,1)`

Main Difference

Manual method calculates new points. OpenGL built-in method transforms the drawing directly.

Important Concept: Homogeneous Coordinates

Your matrix-based programs use points like:

$$[x,\ y,\ 1]$$

Example:

$$[100,\ 100,\ 1]$$

Why 1 is added?

The extra value 1 helps us perform scaling, translation, and rotation using matrix multiplication.

Final Comparison

File	Main Function	Purpose
SimpleSquare.py	`square()`	Draws original square
scale.py	`get_scale2d()`	Changes size
translate2d.py	`get_translate2d()`	Moves object
rotate.py	`get_rotate2d()`	Rotates object
transformation_2d.py	`draw()`	Combines transformations

Final Summary

- A square is created using four vertices.
- Scaling changes the size of the square.
- Translation changes the position of the square.
- Rotation changes the angle of the square.
- Matrix-based programs calculate new coordinates.
- OpenGL built-in functions transform the object directly.

Thank You